STRUKDAT BAB 3

List Ganda (Doubly Linked List)

---

1. Pengertian

Doubly Linked List adalah linked list yang setiap node memiliki dua pointer:

- next → menunjuk node berikutnya
- prev → menunjuk node sebelumnya

Ilustrasi:

```
NULL <- [10] <-> [20] <-> [30] -> NULL
```

Berbeda dengan Singly Linked List yang hanya dapat bergerak ke depan.

---

2. Struktur Node

Implementasi node:

```
struct Node{

    int data;
    struct Node* prev;
    struct Node* next;

};
```

Visualisasi:

```
+------+-------+-------+
| data | prev  | next  |
+------+-------+-------+
```

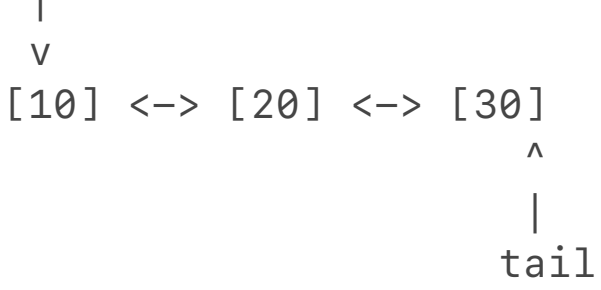
---

3. Head dan Tail

Biasanya digunakan dua pointer:

```
struct Node* head = NULL;
struct Node* tail = NULL;
```

Ilustrasi:



---

4. Membuat Node Baru

```
struct Node* newNode =
(struct Node*) malloc(sizeof(struct Node));
```

Mengisi node:

```
newNode->data = 50;
newNode->prev = NULL;
newNode->next = NULL;
```

---

5. Traversal Maju

Traversal dari head menuju tail.

```
void printForward(struct Node* head){
    while(head != NULL){
        printf("%d ", head->data);
        head = head->next;
    }
}
```

Contoh:

```
10 20 30 40
```

---

6. Traversal Mundur

Traversal dari tail menuju head.

```
void printBackward(struct Node* tail){
    while(tail != NULL){
        printf("%d ", tail->data);
        tail = tail->prev;
    }
```

```
}
```

Contoh:

```
40 30 20 10
```

---

7. Insert First

Awal:

```
10 <-> 20 <-> 30
```

Tambah:

```
5
```

Hasil:

```
5 <-> 10 <-> 20 <-> 30
```

Implementasi:

```
newNode->next = head;

head->prev = newNode;

head = newNode;
```

---

8. Insert Last

Awal:

```
10 <-> 20 <-> 30
```

Tambah:

```
40
```

Hasil:

```
10 <-> 20 <-> 30 <-> 40
```

Implementasi:

```
tail->next = newNode;

newNode->prev = tail;

tail = newNode;
```

---

9. Insert After

Awal:

```
10 <-> 20 <-> 30
```

Sisipkan:

```
25
```

setelah:

```
20
```

Hasil:

```
10 <-> 20 <-> 25 <-> 30
```

Implementasi:

```
newNode->next = curr->next;

newNode->prev = curr;

curr->next->prev = newNode;

curr->next = newNode;
```

---

10. Delete First

Awal:

```
10 <-> 20 <-> 30
```

Hasil:

```
20 <-> 30
```

Implementasi:

```
head = head->next;

head->prev = NULL;
```

---

11. Delete Last

Awal:

```
10 <-> 20 <-> 30
```

Hasil:

```
10 <-> 20
```

Implementasi:

```
tail = tail->prev;

tail->next = NULL;
```

---

12. Delete Node Tertentu

Awal:

```
10 <-> 20 <-> 30 <-> 40
```

Hapus:

```
30
```

Hasil:

```
10 <-> 20 <-> 40
```

Implementasi:

```
curr->prev->next = curr->next;

curr->next->prev = curr->prev;

free(curr);
```

---

13. Searching

Contoh:
Cari:
40

Implementasi:

```
int pos = 1;

while(curr != NULL){

    if(curr->data == x){

        return pos;

    }

    pos++;

    curr = curr->next;

}
```

---

14. Menghitung Jumlah Node

```
int count = 0;

while(curr != NULL){

    count++;

    curr = curr->next;

}
```

---

15. Mencari Nilai Maksimum

```
int maks = head->data;

---

while(curr != NULL){
    if(curr->data > maks){
        maks = curr->data;
    }
    curr = curr->next;
}

---
```

16. Mencari Nilai Minimum

```
int min = head->data;

---

while(curr != NULL){
    if(curr->data < min){
```

```
        min = curr->data;
    }
    curr = curr->next;
}
```

---

17. Kompleksitas Operasi

Operasi	Kompleksitas
Insert First	O(1)
Insert Last	O(1)
Delete First	O(1)
Delete Last	O(1)
Search	O(n)
Traversal	O(n)

---

18. Kesalahan Umum

Lupa Mengubah prev

Salah:

```
curr->next = newNode;
```

tetapi:

```
newNode->prev
```

tidak diubah.

Lupa Mengubah next

Salah:

```
newNode->prev = curr;
```

tetapi:

```
curr->next
```

tidak diperbarui.

Dangling Pointer

Salah:

```
free(curr);

curr->next = ...
```

Mengakses memori yang sudah dibebaskan.

Tail Tidak Diperbarui

Saat menghapus node terakhir.

---

19. Implementasi Lengkap

```
#include <stdio.h>
#include <stdlib.h>

struct Node{

    int data;
    struct Node* prev;
    struct Node* next;
```

```
};

struct Node* head = NULL;
struct Node* tail = NULL;

void insertLast(int data){

    struct Node* newNode =
    (struct Node*) malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;

    if(head == NULL){

        head = tail = newNode;
        return;

    }

    tail->next = newNode;

    newNode->prev = tail;

    tail = newNode;

}

void printForward(){

    struct Node* curr = head;

    while(curr != NULL){

        printf("%d ", curr->data);
```

```
        curr = curr->next;

    }

}

int main(){

    insertLast(10);
    insertLast(20);
    insertLast(30);

    printForward();

    return 0;
}
```

___